Autonomous Maze Navigation of a ROSbot in Webots

A Common Controller Architecture with Per-Map Configuration

Autonomous Robots – Modularbeit, OTH Amberg-Weiden

July 10, 2026

Abstract

This document specifies the navigation approach adopted for the Autonomous Robots Modularbeit. The objective of every maze world is identical: a ROSbot must reach the **blue** pillar first and the **yellow** pillar second, in minimum simulation time, without ever touching a wall or crossing the **green** poison floor, using no Supervisor and no edits to the robot or world. The approach reuses a single, algorithmically fixed navigation controller across all five maze worlds; only the geometry and tuning constants held in each package's `config.py` differ between mazes. The pipeline couples lidar log-odds SLAM, a persistent depth-obstacle layer for lidar-blind hazards, HSV-based visual target acquisition, frontier-driven exploration, A* global planning on a centre-seeking costmap, and a Dynamic Window Approach (DWA) local planner that keeps wall clearance immune to pose drift. The algorithmic pattern is adapted from a public reference implementation, which is credited explicitly to preserve academic integrity.

1 Problem Statement and Design Principle

The task defines a fixed mission over an unknown static maze: navigate from the start pose to the blue pillar, then to the yellow pillar, in the least simulation time, subject to two hard safety constraints — no wall contact and no traversal of the green poison patch. The pillar positions are not provided; the target must be discovered by on-board vision during the run.

The guiding design principle of the approach is *one algorithm, many configurations*. Rather than authoring a separate controller per maze, the project maintains a single navigation stack whose control logic, state machine, sensor interface and observability are identical everywhere. Each maze world binds to its own package copy so that the controller remains self-contained and runs standalone in Webots, but the only substantive difference between those copies is the set of tuning constants in `config.py`. This separation keeps the algorithm auditable and regression-tested once, while allowing the geometry of each arena — grid resolution, arena extent, hazard height bands, sensor complement — to be expressed declaratively as data. The class name (`NavigationController`), the Webots device layer (`RobotInterface`) and the observability layer are shared verbatim across every maze.

2 System Architecture

The controller is decomposed into single-responsibility modules that are executed once per simulation tick. The module layout is uniform across mazes:

Module	Responsibility
`robot_io.py`	`RobotInterface` — the only Webots-facing module: device discovery, guarded reads of motors, lidar, RGB and depth cameras, and IR range-finders.
`odometry.py`	Wheel-encoder and IMU-yaw fusion for the incremental pose estimate.
`mapping.py`	Log-odds occupancy grid, scan-matching, costmap generation, and the poison and auxiliary floating-wall layers.
`depth_model.py`	Depth image to thin-scan conversion (per-column hit/clear) for the floating-wall layer.
`perception.py`	HSV detection of the blue/yellow pillars and green-poison projection with a forward reflex.
`frontier.py` / `astar.py`	Frontier extraction and clustering; A* global planning with clearance-aware path simplification.
`local_planner.py`	Arc-length pure-pursuit carrot plus a Dynamic Window Approach local planner.
`mak_02_controller.p`	`NavigationController` — the mission finite-state machine and the main sensing/planning/driving loop.
`observability.py`	Leveled logging, a structured JSONL event log, and per-stage fault barriers (vendored from `common/`).
`config.py`	Single source of truth for every tuning constant; the only file that differs per maze.

A separate `common/` package sits outside every maze folder and is imported by none of the running controllers, so it cannot influence the code that runs in Webots. It provides a uniform `MazeControllerWrapper` that resolves any maze identifier to its `NavigationController` and exposes only the shared method contract, together with the canonical `observability.py` that each controller vendors a copy of.

2.1 Per-tick data flow

Each control cycle proceeds through the following stages, each wrapped in a fault barrier so that a single bad frame is logged and skipped rather than crashing the mission:

1. **Sensing.** Read encoders, IMU, lidar scan, RGB and depth images, and IR ranges through `RobotInterface`.
2. **State estimation.** Advance odometry, then refine the pose by scan-matching the current lidar scan against the accumulated map.
3. **Mapping.** Integrate the lidar scan into the log-odds grid, accumulate depth hits into the auxiliary layer, and project detected poison.
4. **Perception.** Detect the target pillar and estimate its range.
5. **Costmap.** Rebuild the inflated, centre-seeking costmap from the occupancy, auxiliary and poison layers.
6. **Goal selection.** Choose a frontier goal while exploring, or the pillar standoff once the target is visually fixed and reachable.
7. **Planning.** Plan a global A* path to the current goal.
8. **Driving.** Convert the path to a pure-pursuit carrot and let DWA select the executed velocity command from the live lidar.

3 State Estimation and Mapping

3.1 Pose estimation

The pose is maintained by fusing wheel-encoder odometry with IMU yaw. Because integrated odometry drifts, the estimate is corrected each tick by scan-matching the incoming lidar scan

against the current occupancy map; the matched correction keeps the map self-consistent and, critically, keeps wall clearance referenced to what the robot currently senses rather than to an accumulating error.

3.2 Log-odds occupancy grid

The environment is represented as a 2-D occupancy grid updated in log-odds form from the single-plane lidar. Cells struck by a beam gain occupancy evidence and cells traversed by a beam lose it, so transient noise is averaged out over successive scans. The grid is square and centred on the start pose; its resolution and extent are maze-specific constants (Section 6).

3.3 Poison layer

The green poison floor is a lethal constraint. Green pixels detected by the RGB camera are projected into the grid as a dedicated poison layer that the planner treats as impassable. A complementary forward reflex (`GREEN_REFLEX`) blocks forward motion whenever a cluster of projected green points appears in a narrow corridor directly ahead in the body frame, guarding against a plan that has not yet caught up with a freshly seen patch.

3.4 Auxiliary floating-wall layer

The single-plane 2-D lidar sits at approximately $z = 0.10$ m and is therefore blind to walls that float above, tilt across, or sit below that plane; such walls are marked *free* by the lidar and would otherwise be driven into. The depth camera can see them, but only beyond its minimum range of roughly 0.6 m, which leaves a "dead shell" closer than 0.6 m where a floating wall is invisible to every sensor. The only defence in that shell is memory.

The approach therefore back-projects every depth pixel that falls inside the collision height band [`AUX_Z_MIN` , `AUX_Z_MAX` = robot height] and accumulates per-cell confidence wherever the lidar is blind. A cell becomes lethal "aux" once it reaches `AUX_MIN_HITS` ; a clear sight-line only *decays* that confidence, and the decay ray starts at the depth minimum range so a wall already inside the dead shell is never forgotten during the approach. A lidar-blind gate prevents ordinary full-height walls from being duplicated into the aux layer, and the confidence saturates at `AUX_HIT_CAP` so a confirmed wall survives the few stray clear frames of a close approach. Panels taller than the robot (pass-under bridges) fall outside the height band and stay drivable. Aux cells feed both the A* costmap (as walls) and the DWA obstacle set.

4 Perception and Target Acquisition

Pillars are acquired visually rather than from any hardcoded coordinate. The RGB image is thresholded in HSV space for the blue and yellow pillar hues; the detected blob, combined with the known pillar height, yields a bearing and a range estimate that place the target in the map frame. A running mean over the most recent detections stabilises the estimate against single-frame noise, and successive observations from different viewpoints sharpen it. No world coordinate of any pillar is present in the runtime path; the tabulated positions in the controller documentation are provided only for verification.

5 Exploration, Planning and Control

5.1 Visual-frontier exploration

While the target pillar has not yet been localised, the robot explores the unknown space by frontier search. Frontiers — the boundaries between known-free and unknown cells — are extracted and clustered, and each candidate is scored by a utility that rewards information gain and penalises path length, biased to keep the robot moving forward. The chosen frontier becomes the A* goal.

Exploration and goal acquisition are blended: the instant the RGB camera fixes the target pillar *and* A* finds a path to its standoff, the state machine abandons exploration and commits directly to the pillar (`_maybe_go`). This "visual frontier plus DWA" blend avoids exhaustive mapping — the maze is only explored as far as is necessary to see the next target.

5.2 A* on a centre-seeking costmap

The global planner is an 8-connected, weighted A* over a costmap derived from the occupancy, auxiliary and poison layers. Two properties matter for the tight maze gaps. First, obstacle inflation is footprint-accurate: the lethal radius `HARD_OBS_DIST` equals the true passage half-width of the ROSbot (0.116 m), so A* never routes the wheels into a wall while still leaving a multi-cell free band down the centre of a corridor. Second, cost decreases smoothly with clearance out to `CENTER_PREF_RANGE`, so the planned path rides the medial axis of every corridor. Path simplification is clearance-aware: it shortcuts only through genuinely open cells and never straightens a centred path back against a tight wall. Poison cells are weighted rather than merely forbidden, so the planner skirts them but retains a route if one is unavoidable.

5.3 Pure-pursuit carrot and DWA local planner

The A* path only decides *where* to aim. The executed velocity command is chosen every tick by a Dynamic Window Approach that samples feasible (v, ω) pairs, rolls each out over a short horizon against the *live* lidar obstacles, and rejects any rollout passing closer than `DWA_SAFE_RADIUS`. Surviving candidates are scored by a weighted sum of heading alignment to a pure-pursuit carrot, proximity to that carrot, and clearance (which doubles as corridor centring). Because clearance is evaluated from the current scan rather than the map, wall avoidance is immune to accumulated pose drift. A single floored tight-gap speed factor eases the robot to roughly 0.16–0.26 m/s only where a side wall is genuinely close, preserving cruising speed elsewhere. The four-wheel skid-steer ROSbot is commanded as a two-wheel differential drive, mapping (v, ω) to a left and a right wheel pair; centreline precision comes from the planner, not from any drivetrain modification.

5.4 Mission state machine and recovery

The mission is governed by a finite-state machine:

INIT_SCAN → EXPLORE_BLUE → GO_BLUE → EXPLORE_YELLOW → GO_YELLOW → DONE.

A **RECOVERY** state is reachable from any driving state when forward progress stalls. A windowed-displacement watchdog detects a genuine lack of progress (rather than mere pose jitter) and triggers recovery, which reverses when there is room behind and otherwise spins in place, then re-plans. Repeated recoveries escalate, and an area that cannot be traversed is blacklisted so the planner tries an alternative frontier.

6 Per-Map Configuration

The five maze worlds run the same algorithm; the differences are confined to `config.py`. The table below summarises the substantive per-map settings and the reasoning behind each.

Maze	Grid res.	Sensor set	Rationale for the configuration
Maze1	0.025 m	lidar + depth + IR	Full stack; floating-wall avoidance designed here, aux band from $z = 0.01$ m for near-floor panels.
Maze2	0.025 m	lidar + depth + IR	High-resolution SLAM on a 9.0 m grid; aux band raised to $z = 0.03$ m for the floating/tilted blue-approach cluster.
Maze3	0.025 m	lidar + depth + IR	Map-consistent SLAM fused with map-based poison on an 11.0 m grid.
Maze4	0.04 m	lidar + depth + IR	Reference tuning; coarser grid on an 11.0 m arena, tighter aux max-range.
Maze5	0.05 m	lidar only	Baseline; no floating walls, so the depth-aux and IR layers are unnecessary.

Grid resolution trades runtime against wall sharpness: the tighter arenas with lidar-blind hazards (Maze1–Maze3) use 0.025 m for crisp planning, whereas the open Maze5 baseline uses 0.05 m. The auxiliary-layer height band is set per maze from the actual z-spans of that world's floating, tilted and near-floor panels, and the pillar height constant (0.40 m in Maze2, 0.30 m elsewhere) feeds the vision range estimate. Every geometry constant is read from the world file and documented in the respective controller's README, but no world coordinate is hardcoded in the runtime path.

7 Observability and Validation

The controller carries an enterprise-style observability layer. Operator messages are emitted through leveled, timestamped logging, and a machine- readable trail is written to `maps/run_events.jsonl`: a run-start record with the full device inventory, mission state transitions, pillar-reached splits with timing, recovery entries, status heartbeats, fault records with tracebacks, the final timing table, and a run-end record. Every control-loop stage runs inside a `guarded_stage` barrier, individual device reads are guarded, and a top-level safety net guarantees the wheels are stopped if the controller ever crashes.

The stack is validated without a simulator. Each package ships a `selftest.py` (unit checks over mapping, scan-matching, A*, frontier extraction, DWA, perception, the depth-aux mark/clear logic, and IR lookup, plus a closed-loop "no-crawl" regression) and, where applicable, a `dryrun.py` that drives the real carrot-plus-DWA controller through synthetic straight, narrow-gap, offset, L-corner and pivot corridors and asserts that the robot moves at cruising speed, stays clear of walls, and reaches the goal. A `smoke_test.py` in `common/` confirms that all five mazes resolve and satisfy the shared contract. These Webots-free gates are run after every change, so a tuning adjustment for one maze cannot silently regress the shared algorithm.

8 Attribution and Academic Integrity

The navigation pattern — lidar SLAM, frontier-based exploration, and a `move_base` / `gmapping`-style A*-plus-local-planner stack — follows the publicly available reference implementation **Frontier_Exploration** by Hanwen Cao,[1] which the project adapts and re-implements in Python for the Webots ROSbot. The reference is cited explicitly to make the provenance of the algorithmic pattern transparent and to avoid any appearance of plagiarism. The mapping, perception, depth-auxiliary floating-wall layer, costmap centring, mission state machine, observability, and per-maze configuration described above are original work built for this Modularbeit; the public repository is credited as the source of the underlying exploration-and-planning pattern rather than of the specific code.

[1] `https://github.com/HanwenCao/Frontier_Exploration`

9 Conclusion

The approach delivers the mission — reach blue, then yellow, in minimum time without wall contact or poison traversal — with a single, regression-tested navigation algorithm shared across all five maze worlds and specialised to each only through declarative configuration. Robustness to the mazes' lidar-blind hazards is provided by a persistent depth-obstacle memory; tight-gap safety is provided by footprint-accurate inflation, a centre-seeking costmap, and a drift-immune DWA local planner; and correctness is safeguarded by a structured event log and Webots-free regression gates. The common-controller design keeps the algorithm auditable once while remaining adaptable to any new maze by editing configuration alone.